PES UNIVERSITY

Department of Computer Science and Engineering

JACKFRUIT PHASE 2

PROJECT PLAN DOCUMENT

Anti-Performance Throttling Booster for Applications and Gaming Systems

Project ID	50
Team	Infinite Loop
Team members	Hamsa M N; Harshitha Vangaveti; K Ashritha Reddy; Ankita Vellara
Course	UE24CS341A — Software Engineering, V Semester
Planning horizon	8 weeks: 7 September–30 October 2026
Version	1.0 — Proposed project plan

Prepared for the JACKFRUIT Software Engineering mini-project deliverable.

1. Project Overview

Purpose. This plan defines the lifecycle, tools, deliverables, work breakdown structure (WBS), effort estimates and schedule for Project 50, Anti-Performance Throttling Booster for Applications and Gaming Systems. It is based on the submitted SRS and the JACKFRUIT Phase-2 project-plan requirements.

Objective. Build a Windows-oriented desktop tool that monitors CPU, memory, process and relevant network usage; detects conditions associated with performance degradation; presents alerts and recommendations; supports user-controlled performance profiles; and applies only permitted, user-confirmed optimization actions.

Scope of implementation. The system is a monitoring and assisted-optimization tool. Detection results must be described as suspected throttling when evidence is not conclusive. Critical operating-system processes must be protected, and potentially disruptive actions must require user confirmation.

1.1 Functional scope from the SRS

Feature area	Planned capability	SRS requirements
Real-time monitoring	Start/stop monitoring; CPU/RAM and process metrics; periodic refresh.	REQ-01–REQ-04
Throttling detection	Evaluate configured criteria; report suspected events with timestamps and relevant metrics.	REQ-05–REQ-08
Process analysis and optimization	List process usage; identify eligible background processes; require confirmation; report action outcome.	REQ-09–REQ-12
Network monitoring	Display usage statistics, associate them with the monitoring period and alert on configured thresholds.	REQ-13–REQ-15
Performance profiles	Provide predefined profiles, profile selection, supported edits and profile-aware decisions.	REQ-16–REQ-19
Alerts, recommendations and logs	Display relevant alerts and recommendations; log events/actions; show recent logs.	REQ-20–REQ-23

2. Lifecycle Model and Justification

Selected lifecycle: Agile iterative development with two prescribed implementation sprints. This model is selected because the JACKFRUIT guidelines require Agile methodology, active backlog management and incremental delivery. The eight-week schedule includes requirements and design preparation followed by the course-defined Sprint 1 and Sprint 2 dates.

The team will maintain a prioritized GitHub backlog, assign story points and owners, implement work on feature branches, raise pull requests, run automated checks, review changes and integrate tested modules. Each sprint should end with a working product and demonstrable evidence.

Reason for suitability: the monitoring, detection, profiles, optimization and logging modules can be developed and tested independently, then integrated incrementally. Early integration exposes interface issues before the final validation period. The process also supports individual contribution tracking through commits and pull requests.

3. Tools and Infrastructure

Purpose	Selected tool	Use in this project
Programming language	Python	Monitoring, detection, profile logic, optimization safeguards and logging.
Desktop GUI	Tkinter	Dashboard, controls, metric displays, alerts and log view.
System metrics	psutil	Supported CPU, memory, process and network statistics.
Version control	Git	Branches, commits and merges.
Repository / collaboration	GitHub	Repository, pull requests, Issues and code review.
Agile planning	GitHub Projects	Backlog, story points, assignments, sprint tracking and status.
CI	GitHub Actions	Run tests and configured quality checks on pushes and pull requests.
CI/CD tool specified in course	Jenkins	Use if required by the course/instructor; document its role alongside GitHub Actions.
Unit testing	pytest	Tests for monitoring, detection, profiles, logging and safeguards.

Purpose	Selected tool	Use in this project
Coverage	pytest-cov	Measure and report test coverage; review branch coverage where supported.
Static analysis	SonarQube or approved equivalent	Code-quality and maintainability checks.
Containerization	Docker	Reproducible test/build environment where applicable; not required for desktop runtime itself.
Design/documentation	UML, Markdown, PDF	Architecture, use cases, module design, test evidence and final report.

Tool note: The course guidelines list Git, GitHub, Jenkins, Docker, testing tools, issue tracking/agile planning and static analysis. GitHub Actions is also explicitly required by the Part-2 sprint instructions. The team should configure and demonstrate the tools applicable to the project, and record any tool not used with a clear reason or instructor-approved alternative.

4. Deliverables: Reuse and Build Classification

Existing tools and libraries are classified as reuse/configuration. Project-specific logic, integration, tests and documentation are classified as build. Reusing a library does not replace implementation of the project's required functionality.

Deliverable/component	Classification	Justification
Python runtime and standard libraries	Reuse	Existing language/runtime functionality.
Tkinter	Reuse	Provides GUI widgets; project-specific dashboard and interactions are built.
psutil	Reuse	Provides supported operating-system metrics and process information.
Git, GitHub and GitHub Projects	Reuse/configure	Existing source-control, collaboration and backlog infrastructure.
GitHub Actions / Jenkins	Reuse/configure	Existing automation platforms; team configures project-specific workflows.
pytest, pytest-cov, static analysis tools	Reuse/configure	Existing quality tools; the team builds test cases and configures checks.
Monitoring engine and refresh workflow	Build	Project-specific coordination, sampling, and presentation of metrics.
Throttling-detection logic	Build	Project-specific criteria and suspected-event classification.
Process eligibility and safety checks	Build	Project-specific protection rules and confirmation workflow.
Profile management and optimization controller	Build	Project-specific profile settings, guarded actions and outcome reporting.
Network metrics presentation	Build	Project-specific display, thresholds and monitoring-period association.
Alerts, recommendations and event/action logger	Build	Project-specific rules, timestamps and history view.
Tkinter dashboard and module integration	Build	Project-specific screens, controls and integrated user workflow.
Test suite, traceability matrix and final report	Build	Team-produced tests, actual results, evidence and requirement mappings.
Sprint videos and demonstration material	Build	Team-produced evidence of working sprint outcomes.

5. Work Breakdown Structure (WBS)

The WBS decomposes the project into manageable work packages. Effort is estimated in person-days (PD). The baseline below totals 140 PD, equivalent to 7.0 person-months assuming 20 working days per person-month.

WBS ID	Work package	Principal tasks / output	Effort (PD)
1.0	Project planning and requirements	Scope, SRS mapping, backlog, acceptance criteria, story points, risk list.	8
2.0	Architecture and design	Module interfaces, architecture/use-case diagrams, data structures, UI flow and safety rules.	10
3.0	Monitoring and process analysis	Start/stop monitoring, CPU/RAM sampling, process listing and resource usage.	22
4.0	Throttling detection	Detection criteria, suspected-event classification, timestamps and metric context.	12
5.0	Profiles and optimization	Predefined profiles, supported settings, confirmation, permission checks and action results.	18
6.0	Network, alerts and logging	Network usage statistics, threshold alerts, recommendations, event/action logs and log view.	14
7.0	GUI and integration	Tkinter dashboard, controls, module integration and error/status presentation.	16
8.0	Testing and validation	Unit, integration and system tests; negative tests; defect fixes; traceability and coverage.	22

WBS ID	Work package	Principal tasks / output	Effort (PD)
9.0	CI and code quality	GitHub Actions, test automation, coverage reporting and static/security checks.	8
10.0	Documentation and release	Final report, screenshots, test results, sprint videos, presentation and release freeze.	10
	TOTAL	Baseline estimate	140 PD / 7.0 PM

6. Effort Estimation

Person-month estimate = estimated person-days ÷ 20 working days. The 140 PD total is a rough effort estimate for all team members combined, not an assertion of actual time already spent.

Activity group	Effort (PD)	Effort (PM)
Requirements and planning	8	0.40
Architecture and design	10	0.50
Monitoring and process analysis	22	1.10
Throttling detection	12	0.60
Profiles and optimization	18	0.90
Network, alerts and logging	14	0.70
GUI and integration	16	0.80
Testing and validation	22	1.10
CI and code quality	8	0.40
Documentation and release	10	0.50
TOTAL	140	7.00

Capacity check: Four team members × five working days × eight weeks = 160 person-days of theoretical capacity. The 140 PD estimate leaves 20 PD of capacity for coordination, integration delays and unplanned defects. Update the estimate if actual work logs indicate a different effort.

7. Team Responsibilities and Ownership

Member	Primary functional ownership	SRS requirement mapping
Member	Primary functional ownership	SRS requirement mapping
Hamsa M N	Monitoring engine; CPU/RAM metrics; start/stop workflow and metric interface.	REQ-01–REQ-04
K Ashritha Reddy	Throttling detection; suspected-event reporting; process analysis and eligibility checks.	REQ-05–REQ-10
Harshitha Vangaveti	Performance profiles; optimization safeguards; permission/error handling and action results.	REQ-11–REQ-12, REQ-16–REQ-19
Ankita Vellara	Dashboard integration; network metrics; alerts, recommendations and log presentation.	REQ-13–REQ-15, REQ-20–REQ-23

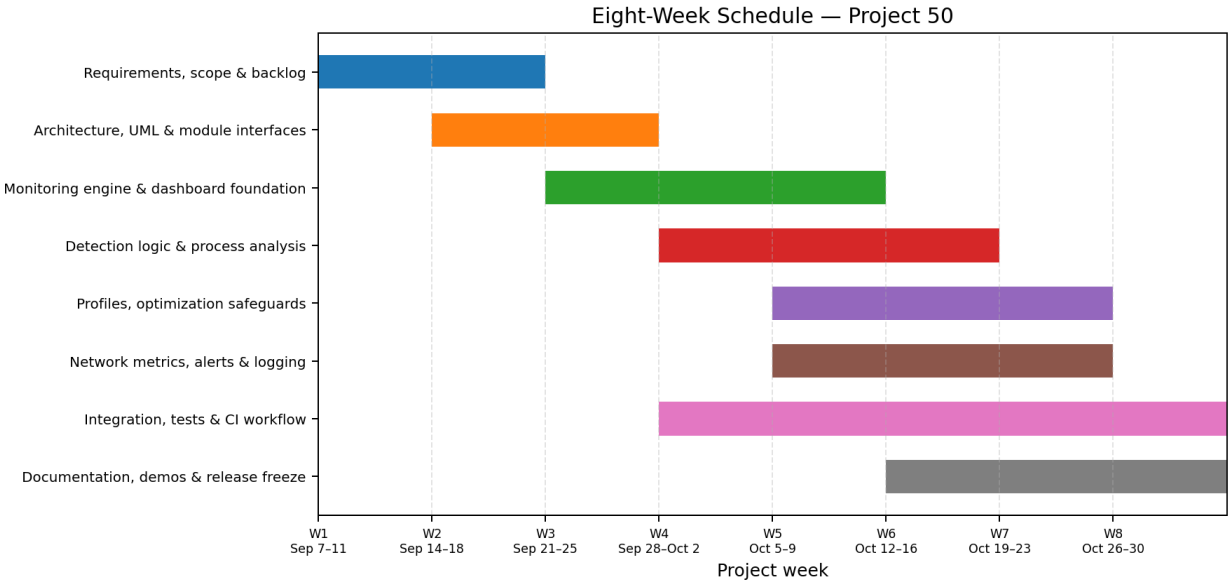
All members are responsible for tests related to their modules, meaningful commits, pull requests, responding to review comments, integration and being able to explain their implementation. Repository owners review and merge PRs. Ownership above is a proposed allocation and should be reconciled with any existing GitHub assignments.

8. Eight-Week Schedule

Week / dates	Planned activities	Milestone / deliverable
W1 · Sep 7–11	Confirm scope; review SRS; identify requirements and risks; establish repo and project board.	Project scope, initial backlog and ownership.
W2 · Sep 14–18	Architecture, use cases, module interfaces, data structures, UI flow and safety constraints.	Architecture/design baseline.
W3 · Sep 21–25	Implement monitoring foundation and initial Tkinter dashboard; create unit-test structure.	Monitoring prototype.
W4 · Sep 28–Oct 2	Implement initial detection logic; integrate monitoring and dashboard; start automated checks.	Integrated baseline candidate.
W5 · Oct 5–9	Develop process analysis, profiles and optimization safeguards; refine issues and acceptance criteria.	Core module implementations.
W6 · Oct 12–16	Dry run: validate backlog and PR workflow; integrate network, alerts and logs; rehearse demo.	Sprint readiness and dry-run evidence.
W7 · Oct 19–23	Official Sprint 1: complete sprint backlog, run GitHub Actions, review/merge PRs, demonstrate working product.	One-minute Sprint 1 video uploaded to repository.
W8 · Oct 26–30	Official Sprint 2: complete remaining features and documentation; test, record demo and freeze development.	Two-minute Sprint 2 video; final docs; development freeze by Oct 30.

Schedule compliance note: the Part-2 deliverables specify a Sprint 1 dry run during 12–16 October, Sprint 1 during 19–23 October and Sprint 2 during 26–30 October 2026. This eight-week schedule preserves those dates. Earlier weeks represent project planning, design and implementation preparation; they do not replace the two official sprint windows.

9. Gantt Chart


Eight-Week Schedule — Project 50
Requirements, scope & backlog
Architecture, UML & module interfaces
Monitoring engine & dashboard foundation
Detection logic & process analysis
Profiles, optimization safeguards
Network metrics, alerts & logging
Integration, tests & CI workflow
Documentation, demos & release freeze
W1
Sep 7–11
W2
Sep 14–18
W3
Sep 21–25
W4
Sep 28–Oct 2
W5
Oct 5–9
W6
Oct 12–16
W7
Oct 19–23
W8
Oct 26–30
Project week

10. Development Workflow and Quality Gates

1. Create or update a GitHub Issue for each backlog item, with requirement link, acceptance criteria, story points and assignee.

2. Implement in a feature branch and commit changes with meaningful messages. Keep module interfaces stable and document interface changes.

3. Raise a pull request. Automated checks should run tests and configured quality checks on the PR.

4. A repository owner reviews the code, tests, safety implications and requirement coverage before merging.

5. After merge, run integration checks and update the test evidence, traceability matrix and sprint status.

Quality gates: passing tests for changed functionality; review approval; no known critical safety regression; protected-process safeguards; explicit user confirmation for potentially disruptive actions; clear handling of permission failures; and traceability from requirement to code and test.

11. Testing and Validation Plan

Test level	Scope	Evidence to retain
Test level	Scope	Evidence to retain
Unit testing	Metric sampling, threshold evaluation, profile settings, logger and process-safety checks.	pytest output, test IDs and coverage report.
Integration testing	Monitoring-to-detection; profiles-to-optimization; UI-to-alert/log flows.	Test records, logs and screenshots.
System testing	Start monitoring, inspect metrics, trigger test condition, view alert, select profile and review logs.	End-to-end test results and demo evidence.
Negative testing	Invalid settings, unavailable metrics, denied permissions, rejected action and unexpected process data.	Observed errors, safe-failure evidence and pass/fail status.
Performance testing	Refresh responsiveness, monitoring overhead and application responsiveness while monitoring is active.	Measured observations and test environment.
Security/safety validation	Critical-process exclusions, confirmation before disruptive actions, local data handling and dependency checks.	Checklist, test results and code-review evidence.

The JACKFRUIT Phase-3 instructions require at least 8–10 test cases for each implemented use case, including unit, integration and system test cases. Manual execution must populate Actual Result and Test Result (Pass/Fail). Do not mark a case as passed before execution; record observed behavior and retain evidence for failures.

12. Risks and Mitigation

Risk	Impact	Mitigation
Risk	Impact	Mitigation
OS metrics or permissions vary by machine	Some metrics/actions may be unavailable.	Handle exceptions; show unavailable status; test under normal and restricted permissions.
Unsafe process action	Application or system instability.	Protect critical processes; require confirmation; default to recommendations rather than automatic termination.
Module interface conflicts	Integration delay during Sprint 2.	Agree on interfaces early; integrate continuously; use PR reviews and integration tests.
Monitoring overhead	The tool itself reduces performance.	Keep sampling lightweight; use a defined refresh interval; measure overhead.
Late defects or incomplete tests	Final deliverables delayed.	Reserve Week 8 for stabilization and documentation; freeze scope and prioritize critical defects.
Tool setup or CI failures	Automated checks do not run reliably.	Validate workflow in dry run; retain logs; document approved alternatives.

13. Final Deliverables and Acceptance Criteria

Deliverable	Acceptance criteria
Deliverable	Acceptance criteria

Deliverable	Acceptance criteria
Project plan	Lifecycle, tools, reuse/build classification, WBS, effort estimate and Gantt chart included.
SRS and design documentation	Requirements, architecture/module design and diagrams updated where needed.
GitHub backlog and source code	Issues assigned, story points recorded, PRs reviewed and individual contributions traceable.
Working application	Core implemented features run together; errors and safety constraints are handled clearly.
Test plan and validation report	Required test cases executed; actual results and pass/fail status recorded; coverage evidence retained.
CI/static-analysis evidence	Configured workflows and quality checks have recorded results.
Sprint videos	One-minute Sprint 1 working-product video and two-minute Sprint 2 working-product video uploaded to the repository.
Final report and demo	Consolidated report, screenshots, deviations and a working final demonstration.
Development freeze	Approved final changes merged and no further feature development after the official freeze.

14. Assumptions and Notes

• Windows is the primary development and demonstration environment, as specified in the SRS.

• Python, Tkinter and psutil are the proposed implementation technologies; actual versions and dependencies should be recorded in the repository.

• Optimization is limited by operating-system permissions and must fail safely when an action is not permitted.

• The project plan is a baseline. Record actual completion dates, effort, defects and any deviation from the SRS/SAD/test plan during execution.

• Course-defined sprint dates and deliverables take precedence over any internal planning convenience.

End of Project Plan Document